

TWELFTHKEY CONSULTING

Developer Implementation Brief

Download Lead Capture System — Gated Asset Downloads with Auto-Mapping

Stack: Vercel (Next.js) + Supabase | April 2026 | Powered by the G2P Framework

How to Use This Brief

This document is your complete implementation guide for the TwelfthKey download lead capture system. Read it top to bottom once before writing a single line of code. Every section builds on the previous one.

The accompanying Excel file (TwelfthKey_Download_Form_Developer_Handoff.xlsx) contains four tabs:

- Problem-Service Mapping — the data object that powers the auto-populate logic
- Form Spec — every field, validation rule, and UX behaviour
- Email Specs — exact copy and structure for both outgoing emails
- CTA Routing Map — every asset tile on the site, current vs. new behaviour

This Word document tells you HOW to build it on Vercel + Supabase. The Excel tells you WHAT to build.

1. What You Are Building

Every downloadable asset on twelfthkey.com (36 templates, 4+ playbooks, 8 dashboard packs) currently routes to either a Discovery Call calendar or a generic Contact Us form. Both are wrong for download intent. You are replacing all of these with a single, unified gated download system.

1.1 The User Journey

- User lands on any resource page (playbooks, templates, dashboards, process maps)
- Clicks 'Download' CTA on any asset tile
- A modal form appears — inline overlay, no page redirect
- User fills 5 fields: Name · Email · Phone · Company · Problem Statement (dropdown)
- On selecting Problem Statement — Service Line and Sub-Service auto-populate below the dropdown (read-only, styled card)
- User clicks Submit / Download
- File downloads immediately in browser
- User sees a success message in the modal for 3 seconds, then modal closes
- Client receives a thank-you email with backup download link + Discovery Call CTA
- TwelfthKey receives an internal notification with all 5 fields + asset downloaded + mapped service

1.2 What This System Does for the Business

- Captures every download as a qualified lead with contact details
- Auto-maps their problem to a service — sales team knows exactly what to pitch before making contact
- Creates a perception moment — user sees their problem reflected back as a service recommendation
- Builds the lead database for follow-up campaigns, CRM enrichment, and sales outreach
- Replaces 3 broken CTA paths (calendar, Contact Us form, no gate) with one consistent system

2. Tech Stack & Architecture

2.1 What You Have

- **Next.js deployed on Vercel**Frontend:
- **Supabase (PostgreSQL)**Database:
- **To be confirmed — see Section 4**File hosting:
- **To be set up — see Section 5**Email service:

2.2 Architecture Overview

The system has 4 components. Build them in this order:

#	Component	What it does	Built in
1	Supabase leads table	Stores every form submission as a lead record	Supabase SQL
2	Next.js modal form component	Renders the form, handles dropdown logic, auto-populate	Next.js / React
3	Next.js API route — /api/download	Validates form, writes to Supabase, triggers emails, returns download URL	Next.js API route
4	Email triggers	Sends client thank-you + internal notification on submission	Resend or Nodemailer

3. Supabase Setup

3.1 Create the Leads Table

Run this SQL in your Supabase SQL editor:

```
CREATE TABLE public.download_leads (  
  id                uuid DEFAULT gen_random_uuid() PRIMARY KEY,  
  created_at        timestampz DEFAULT now(),  
  name              text NOT NULL,  
  email             text NOT NULL,  
  phone             text NOT NULL,  
  company_name      text NOT NULL,  
  problem_statement text NOT NULL,  
  service_line      text NOT NULL,  
  sub_service       text NOT NULL,  
  asset_identifier  text NOT NULL,  
  asset_category    text NOT NULL,  
  source_url        text,  
  is_duplicate      boolean DEFAULT false  
);
```

3.2 Row Level Security

Enable RLS on the table. Allow INSERT from anon (public form submissions). Deny SELECT/UPDATE/DELETE from anon. Only authenticated service role can read data.

Run this after creating the table:

```
ALTER TABLE public.download_leads ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Allow public insert"
  ON public.download_leads FOR INSERT
  TO anon WITH CHECK (true);
```

3.3 Duplicate Detection

- On every INSERT — check if email + asset_identifier combination already exists
- If duplicate found — set is_duplicate = true on the new record. Still insert. Never block a download.
- Do NOT show an error to the user. Process silently.

3.4 Environment Variable

Add to your Vercel project environment variables (Settings → Environment Variables):

```
SUPABASE_URL=your_supabase_project_url
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
```

Use the **SERVICE ROLE** key in the API route (server-side only). Never expose in the browser/client.

4. File Hosting & Download URLs

4.1 Where to Host the Files

Do NOT host downloadable files in the Vercel project repo or public folder. Use one of these:

Option	Service	Cost	Recommendation
A	Supabase Storage	Free tier — 1GB included	✅ Recommended — already in your stack
B	Cloudflare R2	Free up to 10GB/month	Good alternative if volume grows
C	AWS S3	Pay per use	Overkill for current scale
D	Vercel /public folder	Free but no access control	❌ Do not use — files publicly accessible without form

4.2 Using Supabase Storage (Recommended)

- Create a private bucket called twelfthkey-downloads in Supabase Storage
- Upload all files: XLSX templates, PDFs, ZIP packs
- Bucket must be PRIVATE — no public access
- On form submit — API route generates a signed URL with 7-day expiry
- Signed URL is returned to browser (triggers download) AND sent in the client email

API route code pattern:

```
const { data, error } = await supabase.storage
  .from('twelfthkey-downloads')
  .createSignedUrl(filePath, 60 * 60 * 24 * 7); // 7 days

const downloadUrl = data.signedUrl;
```

4.3 File Naming Convention

All files must follow this naming convention exactly (matches the `asset_identifier` in the database):

Category	Filename Pattern	Example
Playbooks	playbooks/[slug].pdf	playbooks/operations-fundamentals-playbook.pdf
SOP Templates	sop-templates/[slug].xlsx	sop-templates/client-onboarding-sop.xlsx
Dashboard Packs	dashboard-packs/[slug].zip	dashboard-packs/operations-dashboard-pack.zip
Governance Frameworks	governance/[slug].xlsx	governance/delegation-of-authority-matrix.xlsx
Process Maps	process-maps/[slug].xlsx	process-maps/pm-003-complaint-to-resolution.xlsx

5. Email Setup

5.1 Recommended Email Service: Resend

Resend is the cleanest option for Vercel + Next.js. Free tier: 100 emails/day, 3,000/month. Setup takes 30 minutes.

- Sign up at resend.com
- Add and verify your sending domain: twelfthkey.com
- Configure SPF, DKIM, and DMARC DNS records (Resend provides exact records)
- Install SDK: `npm install resend`
- Add to Vercel env vars: `RESEND_API_KEY=your_api_key`

5.2 Two Emails Per Submission

Email 1 — Client Thank-You

- To: user's email address from form
- From: hello@twelfthkey.com (display name: TwelfthKey Consulting)
- Subject: Your [asset name] is ready — here's your download link
- Body: Thank you message + Download button (signed URL) + Social links + Book Discovery Call CTA
- Does NOT contain: service mapping, problem statement, any sales language
- See Email Specs tab in the Excel for exact copy

Email 2 — Internal Notification

- To: hello@twelfthkey.com (or your leads inbox)
- From: noreply@twelfthkey.com
- Subject: New Download Lead: [Name] — [Asset Name]
- Body: All 5 form fields + mapped Service + Sub-Service + asset downloaded + source URL + timestamp + pitch prompt
- See Email Specs tab in the Excel for exact structure

5.3 Sending Both Emails from the API Route

Fire both emails in parallel — do not await one before sending the other:

```
await Promise.all([
  resend.emails.send({ to: userEmail, ...clientEmailConfig }),
  resend.emails.send({ to: 'hello@twelfthkey.com', ...internalEmailConfig })
]);
```

6. API Route — /api/download

This is the single server-side handler for all form submissions. Create at: `app/api/download/route.ts`

6.1 Request Payload (POST)

The form sends this JSON body on submit:

```
{
  "name": "Rahul Sharma",
  "email": "rahul@company.com",
  "phone": "9876543210",
  "company_name": "Acme Pvt Ltd",
  "problem_statement": "My operations are chaotic and unscalable",
  "service_line": "Process Excellence", // auto-mapped, sent as hidden field
  "sub_service": "OpEx Structuring", // auto-mapped, sent as hidden field
  "asset_identifier": "[Playbook] Operations Fundamentals Playbook",
  "asset_category": "Playbooks",
  "source_url": "https://twelfthkey.com/consulting/resources/playbooks",
  "file_path": "playbooks/operations-fundamentals-playbook.pdf"
}
```

6.2 Execution Sequence (in order)

- **Step 1 — Validate:** Check all required fields present. Validate email format. Validate phone is 10 digits. Return 400 if invalid.
- **Step 2 — Duplicate check:** Query Supabase: `SELECT id FROM download_leads WHERE email = $1 AND asset_identifier = $2`. Set `is_duplicate` flag.
- **Step 3 — Insert lead:** INSERT all fields into `download_leads` table. Capture the returned id.
- **Step 4 — Generate signed URL:** Call Supabase Storage `createSignedUrl` for the `file_path`. Expiry: 7 days (604800 seconds).
- **Step 5 — Send emails:** Fire both emails in parallel via Resend (see Section 5.3). Pass signed URL to client email.
- **Step 6 — Return response:** Return 200 with `{ success: true, downloadUrl: signedUrl }`. Frontend uses `downloadUrl` to trigger browser download.

6.3 Error Handling

- If Supabase INSERT fails — return 500. Do not send emails. Show error in modal: 'Something went wrong — please try again.'
 - If signed URL generation fails — return 500. Lead is still saved. Log the error.
 - If email sending fails — do NOT return 500. Download still completes. Log the email failure silently.
 - Never block a download because of an email failure. The lead is already captured.
-

7. Frontend: Modal Form Component

7.1 Component Location

Create: components/DownloadModal.tsx (or .jsx if not using TypeScript)

This single component is used on every asset tile across the site. It receives props per asset:

```
type DownloadModalProps = {
  assetIdentifier: string // e.g. '[Playbook] Operations Fundamentals Playbook'
  assetCategory: string // e.g. 'Playbooks'
  assetName: string // Display name shown in modal header
  filePath: string // Supabase storage path
}
```

7.2 Problem-Service Mapping Object

Define this as a constant in the component or a separate lib/mapping.ts file:

```
const PROBLEM_SERVICE_MAP = {
  "My operations are chaotic and unscalable":
    { service: "Process Excellence", subService: "OpEx Structuring" },
  "My team doesn't follow processes consistently":
    { service: "Process Excellence", subService: "OpEx Structuring + SOP Design" },
  "I don't have visibility into what's happening in my business":
    { service: "Analytics & BI", subService: "Analytics Visualization Suite" },
  "My financials are always delayed or unclear":
    { service: "Process Excellence", subService: "Governance Reporting Setup" },
  "I'm spending too much time firefighting":
    { service: "Process Excellence", subService: "Fractional CBO Services" },
  "I want to prepare my business for funding or expansion":
    { service: "Fundraise Support Strategy", subService: "Investor Pitch Deck + Financial Model" },
  "I'm losing clients due to inconsistent delivery":
    { service: "Process Excellence", subService: "OpEx Structuring + Delivery Quality Gates" },
  "Other":
    { service: "Process Excellence", subService: "Operational Health Index Report" },
};
```

7.3 Auto-Populate Behaviour

- When Problem Statement dropdown value changes — look up PROBLEM_SERVICE_MAP[value]
- Set serviceLine and subService state variables
- Render a styled card below the dropdown with fade-in animation (200ms opacity transition)
- Card label: 'Based on your challenge, here's where we can help:'
- Service line: teal (#0E7C7B), bold, 14px. Sub-service: grey (#4A4A4A), 12px, italic
- Card background: #E8F4F4 (light teal). Border radius: 8px. Padding: 12px.
- Fields are display-only — not form inputs. Pass values as hidden inputs in the form.

7.4 Submit Flow

- On submit: show loading spinner on button, disable all fields
- POST to /api/download
- On success: browser window.open(downloadUrl, '_blank') OR anchor.click() pattern
- Show success message in modal: 'Your download has started. Check your email for a backup link.'
- Auto-close modal after 3 seconds
- On error: show inline error message, re-enable form, keep modal open

7.5 Mobile Behaviour

- On screens < 768px: modal becomes a bottom sheet (fixed bottom, full width)

- Handle: 4px grey bar centred at top. Dismiss on swipe down or X button.
- Keyboard open: sheet scrolls up. Submit button must always be visible above keyboard.
- Use CSS: position: fixed; bottom: 0; left: 0; right: 0; border-radius: 16px 16px 0 0;

8. CTA Changes on Existing Pages

All existing CTAs across the resources section must be updated. Full list is in the CTA Routing Map tab of the Excel.

8.1 CTA Label Changes

Page / Category	Old CTA Label	New CTA Label
Playbooks page — all tiles	Request access	Download Playbook
SOP Templates — all tiles	Browse → calendar redirect	Download Template
Dashboard Templates — all tiles	Browse → calendar redirect	Download Dashboard Pack
Governance Frameworks — all tiles	Browse → calendar redirect	Download Framework
Process Maps — all tiles	Browse → calendar redirect	Download Process Map

8.2 What to Remove

- Remove all routing to the Discovery Call calendar page from any template/playbook/dashboard CTA
- Remove all routing to the Contact Us form from Playbooks 'Request access' buttons
- The Discovery Call calendar remains accessible via the 'Book Discovery Call' button in the navbar and in the client thank-you email — just not as the download CTA

9. Vercel Environment Variables

Add all of the following in Vercel Dashboard → Project → Settings → Environment Variables. Set for Production + Preview + Development.

Variable Name	Value	Notes
SUPABASE_URL	https://[your-project].supabase.co	From Supabase project settings
SUPABASE_SERVICE_ROLE_KEY	eyJ...	Service role key — never expose client-side
SUPABASE_ANON_KEY	eyJ...	Anon key — safe for client-side use if needed
RESEND_API_KEY	re_...	From Resend dashboard after domain verification
NEXT_PUBLIC_SITE_URL	https://twelfthkey.com	Used to build source URLs in internal email
INTERNAL_NOTIFICATION_EMAIL	hello@twelfthkey.com	Recipient of internal lead notification
DOWNLOAD_LINK_EXPIRY_SECONDS	604800	7 days — signed URL expiry for Supabase Storage

10. Build Sequence — Do It in This Order

Do not skip ahead. Each step depends on the previous one being working.

Step	Task	What to do	Est. Time
Step 1	Supabase table + RLS	Create download_leads table. Run RLS policy. Test INSERT from Supabase dashboard.	~1 hr
Step 2	Supabase Storage bucket	Create private twelfthkey-downloads bucket. Upload 2–3 test files.	~30 min
Step 3	Vercel env variables	Add all 7 env vars. Redeploy to confirm they load.	~30 min
Step 4	Resend domain verification	Add DNS records, verify sending domain. Send a test email.	~1 hr
Step 5	/api/download route	Build the API route. Test with Postman/curl — verify Supabase insert + signed URL + emails all fire.	~2–3 hrs
Step 6	DownloadModal component	Build the modal with form, dropdown, auto-populate, submit handler, success state.	~3–4 hrs
Step 7	Wire up first asset tile	Connect one tile (e.g. Operations Fundamentals Playbook) to the modal. Test end-to-end on staging.	~1 hr
Step 8	Mobile testing	Test bottom sheet behaviour on iOS Safari and Android Chrome. Fix keyboard/scroll issues.	~1 hr
Step 9	Roll out to all tiles	Update all remaining CTAs across all 5 resource categories (see CTA Routing Map in Excel).	~2–3 hrs
Step 10	QA + go live	Test every asset. Verify email delivery, download links, Supabase records, duplicate handling.	~2 hrs

11. Acceptance Criteria — How to Know It's Done

Every item below must pass before this goes live:

- Clicking 'Download' on any asset tile opens a modal — no page redirect
- Selecting a Problem Statement auto-populates Service Line and Sub-Service in a styled read-only card with fade-in animation
- Form cannot be submitted without all 5 fields filled. Email and phone validation working.

- On submit — file downloads immediately in browser
- Success message appears in modal for 3 seconds, then modal closes
- Supabase download_leads table has a new row with all fields correctly populated
- Client receives thank-you email within 60 seconds — contains download link, social icons, Book Discovery Call CTA
- Client email does NOT contain service mapping, problem statement text, or any sales language
- TwelfthKey receives internal notification email with all 5 fields + mapped service + asset downloaded + source URL
- Download link in client email works and delivers the correct file
- Submitting the same email + asset twice — duplicate is flagged in DB, download still works, no error shown to user
- Mobile: modal is a bottom sheet. Form is usable with keyboard open. Submit button visible.
- All CTA buttons across Playbooks, SOP Templates, Dashboard Templates, Governance Frameworks, and Process Maps trigger the modal — none route to calendar or Contact Us form
- Dashboard tiles deliver a ZIP file containing all dashboards in that category